Name: **Joshua Ngui**
Date: **5/1/2025**
Class: **CSC 4320**
CRN: **17449**

# Project 2 Report

## Overview

For this project, I chose to solve the producer-consumer problem, where producer threads adds items to a buffer and consumer threads remove items.

## Implementation Details

Before doing any thread scheduling, my program first obtains thread info from a user-specified file and stores that info in an array of `ProcessThread` objects, which, among other things, contains each thread's PID, arrival time, burst time, priority value, and signifier of whether or not said thread is a producer or consumer thread. These threads are then started via the `Thread.start()` function one after the other by looping through the array. Arrival times and CPU burst times were simulated using the `Thread.sleep()` function. To keep only one producer and consumer thread from running at a time, I utilized `Semaphore` objects. Namely, I created `Semaphore` objects named `canRead` and `canWrite` that each have one permit, so that only the first consumer and producer thread can, respectively, acquire a permit from them, forcing other thread to block until the first threads release permits back to the semaphores. Additionally, I created `Semaphore` objects named `full` and `empty` to simulate the `f` and `e` semaphores typically used to track the number of full and empty buffer slots. These semaphores prevent consumer threads from reading from empty slots or producer threads form overwriting full slots.

Name: **Joshua Ngui**
Date: **5/1/2025**
Class: **CSC 4320**
CRN: **17449**

# Results

My tests for my program utilized the following processes.txt file:

| PID | Arrival_Time | Burst_Time | Priority | Is_Producer |
|---|---|---|---|---|
| 1 | 0 | 5 | 6 | 1 |
| 2 | 3 | 3 | 2 | 1 |
| 3 | 1 | 2 | 5 | 1 |
| 4 | 5 | 3 | 1 | 1 |
| 5 | 2 | 6 | 3 | 1 |
| 6 | 4 | 9 | 7 | 1 |
| 7 | 0 | 5 | 4 | 1 |
| 8 | 4 | 3 | 4 | 0 |
| 9 | 3 | 2 | 7 | 0 |
| 10 | 6 | 3 | 2 | 0 |
| 11 | 6 | 5 | 5 | 0 |
| 12 | 2 | 3 | 3 | 0 |
| 13 | 0 | 2 | 6 | 0 |
| 14 | 7 | 3 | 1 | 0 |

Using this data, processes finished in the following order during my sample run:

1 → 13 → 7 → 3 → 12 → 9 → 5 → 8 → 2 → 11 → 6 → 10 → 4 → 14

# Challenges & Solutions

One challenge I encountered was determining how producer threads would generate data for the buffer. The solution I came up with was to utilize the `Math.random()` function to randomly select characters from a premade array and combine them into a string to place into a buffer slot. Another challenge I encountered was how to signify whether a thread was a producer or consumer thread. The solution I devised for this problem was to pass an integer signifying as such during creation of a `ProcessThread` object. If said integer had the value of `1`, an attribute for the object named `isProducer` would be set to `true`; otherwise, it would be set to false.